

Serial Communication Fundamentals and UART

Why one wire beats thirty-two, the six problems every protocol must solve, and the queue that makes it all work without stalling the CPU

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 8, printed pages 237–249 and 257–269.

Lab manual: Lab 02 (UART bring-up), and every lab thereafter that prints anything.

1 What we are actually after this week

You have been using UART since Lab 02 without knowing how it works. This week and the next two explain the whole family.

The question: **how do two chips exchange data over as few wires as possible, when neither can see the other's clock?**

The plan:

1. **Why serial at all**, when parallel is obviously faster.
2. **The six problems** every serial protocol must solve. This framework carries through Weeks 15 and 16, so it earns careful treatment.
3. **Software structure**: why communication is doubly asynchronous, and the **queue** that fixes it.
4. Queue implementation, including the critical sections, which is Week 08 cashing in.
5. **UART**: the frame, parity, and the baud rate generator.
6. The USART peripheral: registers, flags, and **polled versus interrupt-driven** versions of the same program.

Item 2 is the conceptual spine of three weeks. Item 4 is the best worked example of shared-data safety in the whole book. Item 5 produces a calculation question.

2 Deep dive 1: Why serialize

Serial and parallel

Parallel: an organisation in which all parts of an item are simultaneously available or active. A separate wire for every bit.

Serial: an organisation in which parts of an item are sequentially available, but not simultaneously. One wire, one bit at a time.

Inside the MCU, components communicate over parallel buses using the native data type, sending a whole word in a single transaction. Fast, and free, because the wires are microscopic traces on one die.

Off-chip it stops being free, and the costs grow with distance and speed:

- **Pin count.** A 32-bit bus needs 32 data pins, plus address pins, plus read/write control lines. Both chips pay.
- **Package size and cost.** More pins means a bigger, dearer package.
- **Connectors and cable.** More contacts, more wires, more weight.
- **Board routing.** Thirty-two traces that must be length-matched so their bits arrive together.

Serialization reverses all of it: fewer pins, smaller packages, cheaper connectors, lighter cables, simpler boards. Which is why most MCUs support serial communication and why there is an enormous ecosystem of compatible parts.

Beware the trap

The obvious objection is that serial must be slower, since one wire moves one bit while thirty-two move thirty-two.

It is true at equal bit rates and false in practice, because **parallel does not scale to high speed**. The reason is **skew**: at high frequency, small differences in trace length make bits arrive at slightly different times, and the receiver must wait for the slowest. A single serial line has no skew to worry about, so it can be clocked far faster.

This is why the industry moved from parallel ATA to serial ATA, from PCI to PCI Express, from parallel printer ports to USB. **Serial won on speed, not just on cost.**

2.1 Where this sits in the OSI model

The ISO Open Systems Interconnection model defines seven layers. Three matter here:

Layer	Name	Defines
1	Physical	How symbols are represented on the medium, as voltages or currents
2	Data link	Media access control : when a node may transmit. Logical link control : how a receiver finds the start and end of a message, and how errors are detected
3	Network	Addressing, splitting long data across messages, error handling

3 Deep dive 2: The six problems

Every serial protocol solves the same six problems. SPI, UART, and I²C differ in *which answers they pick*, so having the list makes the next two weeks a comparison rather than three separate memorisation tasks.

3.1 Problem 1: Serialization

Serialization and deserialization

Serialization: converting information from parallel to serial form.

Deserialization: the reverse.

Both are done by **shift registers**. The transmitter loads a parallel-in-serial-out register and shifts bits out; the receiver clocks bits into a serial-in-parallel-out register and reads

the parallel output once it is full.

Baud rate (or symbol rate): the rate at which communication symbols are transmitted.

3.2 Problem 2: Symbol timing

For deserialization to work, the receiver must sample the line **once per symbol**, and specifically **in the middle of each symbol time**. Sampling near an edge risks catching a transition, where the signal may be corrupted by noise or by slow circuits.

Two approaches, and this is the deepest fork in the road:

	Synchronous	Asynchronous
Clock	Transmitter's clock wired to receiver	No clock sent; receiver has its own
Wires	Three: clock, data, ground	Two: data, ground
Reliability	High, clock is always correct	Depends on both clocks matching
Examples	SPI, I ² C	UART

The definitions, which recur

Synchronous: activities synchronised with each other, or a protocol that **sends clocking information**.

Asynchronous: activities not synchronised, or a protocol that **does not send clocking information**.

Recall Week 07 used "asynchronous" for interrupts, meaning they can occur anywhere. Same word, same underlying meaning: no shared timing reference.

3.3 Problem 3: Message framing

If the asynchronous receiver has no clock connection, how does it know when a message starts?

Framing symbol

A symbol used to indicate the start or end of a message. The receiver's clock starts running when it detects the framing symbol, such as a **start bit**.

So the start bit is not wasted overhead. It is the entire mechanism by which an asynchronous receiver aligns itself, once per byte.

3.4 Problem 4: Error detection

Links are vulnerable to noise, especially when long or poorly shielded. Common methods: **parity bits**, **checksums**, and **cyclic redundancy checks**. Some protocols include a dedicated error-notification bit letting a receiver tell the transmitter, and everyone else, that a message arrived corrupted.

3.5 Problem 5: Acknowledgments

Acknowledgment

A device response indicating successful reception of a message.

Some protocols put an acknowledgment field in every message (I²C does), some use a separate acknowledgment message, and **some have nothing at all** (UART and SPI). A protocol with no acknowledgment cannot tell you whether anyone was listening.

3.6 Problem 6: Media access control and addressing

Media access control

Rules controlling when a node may transmit on shared media. Without them, two transmitters may send simultaneously, producing a **collision** that garbles both messages.

The common approach: one coordinating node transmits as needed, and the others follow a strict "speak when spoken to" rule. This is called **master/slave**.

Beware the trap

A note the book makes and it is worth repeating. SPI and I²C were created in the 1980s and use "master" and "slave" throughout their device names, signal names, and register bit names. There is an enormous installed base of hardware, documentation, and tools using those terms, so the textbook uses the industry-standard names while asking readers to consider their unintended consequences and to avoid repeating that kind of mistake in future work.

Practically: you will encounter both these and newer alternatives (controller/peripheral, main/subordinate) in datasheets. Recognise both, because the register bit is still called MSTR.

And **addressing**: with multiple possible receivers, the transmitter must indicate which one it means. Either on **separate select signals** (SPI's approach) or **inside the message** (I²C's).

3.7 Development tools

An **oscilloscope** shows voltage over time and leaves you to decode it, which is slow, tedious, and error-prone. A **logic analyser with a protocol decoder** interprets the signals automatically. The book's example decodes a waveform into: a write to device 0x1D of value 0x01, then a read from device 0x1D of six bytes.

In the lab

Get access to a logic analyser before Lab 08 and Lab 09 if you possibly can. Debugging SPI or I²C without one means guessing from the outcome, and the outcome is usually "the sensor returned zero", which is consistent with about nine different faults.

With a decoder you see immediately whether the clock ran, whether your address byte was right, whether the device acknowledged, and whether the data came back. That converts a whole evening into ten minutes. Cheap USB analysers with decoders for UART, SPI, and I²C are widely available, and the book mentions the Digilent Analog Discovery 2 as a combined instrument.

4 Deep dive 3: Why communication needs a queue

Now the software side, and there is a genuine difficulty here.

Communication makes sharing the CPU harder for **two** reasons, and both are about timing:

1. **Reception is asynchronous.** You do not know which instruction will be executing when data arrives. There is no timing relationship between your program's progress and the arrival of bytes.
2. **Transmission is also asynchronous.** Sending one item takes a long time relative to instruction execution, so the program executes many instructions before the transmitter is ready for the next item, and you cannot predict which one.

You could **poll**: spin in a loop until data has been received. Simple, and recall Week 04's whole argument against it.

Better: **interrupts**. The peripheral raises an interrupt when it has received an item, and again when it is ready to send one. But that only relocates the problem, because an ISR must be short (Week 08), and the receiving *task* is not ready to process the byte right now.

The structure, and it is worth memorising as a picture

Receive: byte arrives → **RX ISR** reads it from the peripheral and **enqueues** it into the receive queue → later, a **task dequeues** and uses it.

Transmit: a **task enqueues** data into the transmit queue → peripheral becomes ready → **TX ISR dequeues** and writes it to the peripheral.

Two queues, one per direction. The queue is what **decouples** the hardware's timing from the software's.

Recall Week 08's partitioning discussion, which asked what happens if the ISR runs several times before the task can. For a switch position, latest-value-wins was fine. For serial data, **every byte counts**, and Week 08 explicitly said that case needs a queue. This is that queue.

5 Deep dive 4: Implementing the queue

Listing 1: Book Listing 8.1. Note the comment about volatile.

```

1 | #define Q_MAX_SIZE (256)
2 |
3 | typedef struct {
4 |     uint8_t      Data[Q_MAX_SIZE];
5 |     unsigned int Head;      /* index of oldest element */
6 |     unsigned int Tail;     /* index of next free space */
7 |     unsigned int Size;     /* number of elements in use */
8 | } Q_T;
```

Ring buffer

Rather than shifting all the data on every operation, keep two indexes. **Head** is the oldest element, read by the next dequeue. **Tail** is the free space used by the next enqueue. Both increment and **wrap around to zero** at the end of the array.

Make the array size a power of two. Then wrapping is a bitwise AND with $\text{SIZE} - 1$ instead of a modulus, and recall from Week 10 that division and remainder are expensive while masking is one instruction. The book's code uses %, and notes this optimisation explicitly.

How big should it be? It depends on how often items may be enqueued, how long before the first dequeue, and how often subsequent dequeues happen. There is no universal answer, only an arithmetic one.

5.1 The critical sections, which are the real lesson

The queue is shared between ISRs and main-line code, so **define queue variables as volatile**, exactly as Week 08 required.

And volatile is not enough, exactly as Week 08 warned. The book spells out the failure:

Beware the trap

The lost-update, in the queue's own terms.

Q_Dequeue is running. Size holds N , and is loaded into a register to be decremented to $N - 1$. Before the store happens, a character arrives, the CPU preempts, and the ISR calls Q_Enqueue. That loads N , the *old* value, increments to $N + 1$, and stores it. The ISR completes, Q_Dequeue resumes and stores $N - 1$.

Size should be N . It is $N - 1$. **One received byte has vanished from the accounting**, and the queue will now think it is emptier than it is, permanently.

This is exactly the Week 08 counter trace, in a structure you will actually ship.

The book also names a second flavour: **multiple variables that must be updated together**. If there were no Size field, computing the queue length would need both Head and Tail via $(\text{Tail} - \text{Head}) \% \text{Q_MAX_SIZE}$, so *every* access to either would be a critical section. Keeping an explicit Size field is partly a design choice to reduce that.

Listing 2: Book Listing 8.2. The save-modify-restore pattern from Week 07.

```

1 int Q_Enqueue(Q_T *q, uint8_t d) {
2     uint32_t masking_state;
3     if (!Q_Full(q)) {
4         q->Data[q->Tail] = d;
5
6         masking_state = __get_PRIMASK();    /* save */
7         __disable_irq();                    /* modify */
8         q->Tail = (q->Tail + 1) % Q_MAX_SIZE;
9         q->Size++;
10        __set_PRIMASK(masking_state);        /* restore */
11
12        return 1;                            /* success */
13    }
14    return 0;                                /* queue full */
15 }
16
17 uint8_t Q_Dequeue(Q_T *q) {
18     uint32_t masking_state;
19     uint8_t t = 0;
20     if (!Q_Empty(q)) {
21         t = q->Data[q->Head];
22         q->Data[q->Head] = '_';              /* blank it, for debugging */
23
24         masking_state = __get_PRIMASK();
25         __disable_irq();
26         q->Head = (q->Head + 1) % Q_MAX_SIZE;
27         q->Size--;
28         __set_PRIMASK(masking_state);
29     }
30     return t;
31 }

```

Two details in that code worth copying

The critical section is as short as possible. The array write happens *outside* it; only the index and size updates are protected. Recall Week 08's snapshot-inside, compute-outside rule.

Enqueue returns a status code rather than overwriting when full. Silently dropping the oldest data is a design decision, and making it explicit forces the caller to decide what a full queue means.

5.2 Blocking or not

Do you want the code to wait for data (dequeue) or for space (enqueue)?

Non-blocking: test the condition; if false, defer the work. **Blocking:** loop until the condition is true, which does *not* share the processor. Recall Week 05: inside an FSM the test should be its own state, ending and repeating on the next call, and under a scheduler the code should yield.

Beware the trap

Never perform a potentially blocking operation inside an interrupt handler.

The reason given is precise: it introduces *sporadic* timing delays that are hard to reproduce and therefore hard to debug. An ISR that occasionally waits for queue space has just added an unbounded, unpredictable term to the worst-case latency of every other interrupt in the system.

Instead, the handler should be prepared to **discard the data** or handle it another way, and signal an error to the application. Look at USART2_IRQHandler in Section 6 and you will see it does exactly that: if (!Q_Full(&RxQ)) then enqueue, otherwise drop.

6 Deep dive 5: UART

UART and USART

UART: universal asynchronous receiver/transmitter, a peripheral for asynchronous serial communication.

Since such peripherals usually also support synchronous operation, they are often called **USART**, the S being synchronous. On STM32 the peripheral is a USART used in UART mode when you do not need the clock output pin.

Both transmitter and receiver have clock generators that **must be configured to the same speed**, because none is transmitted.

Unlike SPI, where master and slave transmit simultaneously, a UART's transmission and reception are **independent**, with separate hardware for each.

6.1 The frame

Field	Value	Purpose
Start bit	Always 0	Framing. Lets the receiver's clock synchronise
Data	Typically 8 bits	The payload. LSB or MSB first
Parity	Optional	Error detection
Stop bit(s)	Always 1	Lets the receiver detect timing errors

Parity, defined carefully

Parity is determined by the total number N of one bits in **the data character and the parity bit together**. If N is even the message has even parity; if odd, odd parity.

The transmitter computes the data's parity and **sets the parity bit so the total matches the configured scheme**. The receiver recomputes across data and parity bit and checks. A mismatch raises a parity error for software to handle.

Its limitation, worth knowing: parity detects any **odd** number of bit errors and **misses every even number**. Two flipped bits look perfectly fine. That is why serious links use CRCs.

If a **zero** is received where a stop bit should be, the receiver reports a **framing error**, which usually means the two ends disagree about the baud rate.

6.2 The baud rate generator**Oversampling**

To resist noise, the receiver samples the RX line **multiple times per bit** and takes the most common value.

After reset the factor is **16**, because OVER8 in USARTx_CR1 is zero. Setting OVER8 to one reduces it to 8, which allows a higher baud rate from the same clock at the cost of noise immunity.

With $16\times$ oversampling the arithmetic is pleasantly simple:

$$\text{Baud rate} = \frac{f_{\text{BusClock}}}{\text{USARTx_BRR}} \iff \text{USARTx_BRR} = \frac{f_{\text{BusClock}}}{\text{Baud rate}}$$

Worked: 48 MHz bus clock, 9600 baud gives $\text{BRR} = 48,000,000/9600 = 5000$.

Beware the trap

BRR is a 16-bit integer, so **most baud rates are not exactly achievable**, and the division truncates.

At 48 MHz and 115200 baud, $48,000,000/115200 = 416.67$, which truncates to 416, giving an actual rate of $48,000,000/416 = 115,385$ baud. That is a 0.16% error, which is fine.

The tolerance for a UART is roughly $\pm 2\%$ total between the two ends, because the receiver resynchronises on every start bit and only has to stay aligned for about ten bit times. Beyond that, the sampling point drifts far enough by the stop bit to produce framing errors.

So if your terminal shows consistent garbage rather than nothing, suspect the baud rate first. Garbage means the line is active and the framing is wrong; silence means nothing is transmitting at all. Those are different bugs.

7 Deep dive 6: The USART peripheral

7.1 Registers

Register	Field	Meaning
CR1	M1, M0	Data length: 7 bits (M1=1, M0=0), 8 bits (both 0), 9 bits (M1=0, M0=1)
CR1	PCE	Enable parity generation and checking
CR1	PS	With PCE: even (0) or odd (1) parity
CR1	TE, RE, UE	Enable transmitter, receiver, and the USART itself
CR1	TXEIE, TCIE, RXNEIE, PEIE	Interrupt enables
CR1	OVER8	Oversampling 16 (0) or 8 (1)
CR2	STOP	0.5, 1, 1.5, or 2 stop bits
CR2	MSBFIRST	MSB first (1) or LSB first (0)
CR3	EIE	Enable error interrupts
BRR	—	Baud rate divisor
TDR	—	Transmit data. Writing it clears TXE
RDR	—	Receive data. Reading it clears RXNE

7.2 Status flags in USARTx_ISR

Flag	Kind	Meaning
TXE	TX	Transmit buffer has room for another character
TC	TX	Transmission complete
RXNE	RX	Data available to read from the receive buffer
IDLE	RX	Receive line is idle
PE	Error	Parity error
FE	Error	Framing error
ORE	Error	Overrun: new data arrived before the old was read
NF	Error	Noise detected, via oversampling disagreement

Flags are cleared by writing 1 to the corresponding bit in USARTx_ICR. Recall Week 07's EXTI->PR: same write-1-to-clear discipline, same atomicity reason.

Beware the trap

ORE, the overrun error, is the flag that tells you your software is too slow.

It means a character arrived while the previous one was still sitting unread in RDR, so a byte was lost. Recall the CCyOF overcapture flag from Week 13; this is the same idea for serial data.

Causes, in order of likelihood: your RX ISR is not enabled, some other ISR is hogging the CPU, a critical section is too long, or you are polling with too slow a loop. It is not a wiring fault, it is a scheduling fault, and **on some STM32 parts an unhandled ORE**

blocks further reception until cleared, which turns one lost byte into total silence.

7.3 Initialisation

Listing 3: Book Listing 8.8. The four phases from Week 03, again.

```

1 #define F_USART_CLOCK (48UL*1000UL*1000UL)
2
3 void Init_USART2(void) {
4     /* 1. clocks */
5     RCC->APB1ENR |= RCC_APB1ENR_USART2EN;
6     RCC->AHBENR  |= RCC_AHBENR_GPIOAEN;
7
8     /* 2. route PA2 (TX) and PA3 (RX) to USART2 via AF1 */
9     MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER2, ESF_GPIO_MODER_ALT_FUNC);
10    MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER3, ESF_GPIO_MODER_ALT_FUNC);
11    MODIFY_FIELD(GPIOA->AFR[0], GPIO_AFR_L_AFSEL2, 1);
12    MODIFY_FIELD(GPIOA->AFR[0], GPIO_AFR_L_AFSEL3, 1);
13
14    /* 3. frame format: 9600 baud, 8 data bits, no parity, 1 stop bit */
15    USART2->BRR = F_USART_CLOCK / 9600;
16    MODIFY_FIELD(USART2->CR1, USART_CR1_PCE, 0);
17    MODIFY_FIELD(USART2->CR1, USART_CR1_M, 0);
18    MODIFY_FIELD(USART2->CR2, USART_CR2_STOP, 0);
19
20    /* 4. enable transmitter, receiver, and the peripheral */
21    MODIFY_FIELD(USART2->CR1, USART_CR1_TE, 1);
22    MODIFY_FIELD(USART2->CR1, USART_CR1_RE, 1);
23    MODIFY_FIELD(USART2->CR1, USART_CR1_UE, 1);
24 }

```

"8N1", and how the two ends agree

The shorthand **8N1** means 8 data bits, No parity, 1 stop bit. With a baud rate that gives you the five settings both ends must match: **baud rate, data bits, parity, stop bits, and bit order.**

None of these is negotiated. A UART has no handshake and no acknowledgment, so if the two ends disagree there is no error message, just wrong data or a framing error. This is why every terminal program asks you for "9600 8N1".

7.4 Polled version

Listing 4: Book Listing 8.9. Echo each character incremented by one.

```

1 void UART_Transmit(uint8_t data) {
2     while (!(USART2->ISR & USART_ISR_TXE)) /* wait for room */
3         ;
4     USART2->TDR = data; /* writing clears TXE */
5 }
6
7 uint8_t UART_Receive(void) {
8     while (!(USART2->ISR & USART_ISR_RXNE)) /* wait for data */
9         ;
10    return USART2->RDR; /* reading clears RXNE */
11 }
12
13 /* main loop */
14 while (1) {
15     receive_data = UART_Receive();

```

```

16     transmit_data = receive_data + 1;
17     UART_Transmit(transmit_data);
18 }

```

Simple, and it is Week 04's starter program again: both those while loops are busy-waits that own the CPU.

7.5 Interrupt-driven version

Now the queued structure from Section 3, made real. The demo responds to each character with "You pressed x."

Listing 5: Book Listing 8.13. Note what happens when a queue is full.

```

1 void USART2_IRQHandler(void) {
2     uint8_t ch;
3
4     if (USART2->ISR & USART_ISR_RXNE) {           /* receive buffer not empty */
5         ch = USART2->RDR;                          /* reading clears RXNE */
6         if (!Q_Full(&RxQ)) {
7             Q_Enqueue(&RxQ, ch);
8         }                                           /* else: DROP it, no blocking */
9     }
10
11    if (USART2->ISR & USART_ISR_TXE) {              /* transmit buffer empty */
12        if (!Q_Empty(&TxQ)) {
13            USART2->TDR = Q_Dequeue(&TxQ);          /* writing clears TXE */
14        } else {
15            MODIFY_FIELD(USART2->CR1, USART_CR1_TXEIE, 0); /* nothing left */
16        }
17    }
18 }

```

The self-disabling transmit interrupt, which is elegant

TXE is set whenever the transmit buffer is empty, which is *almost always* when you are not transmitting. So if TXEIE stayed enabled, the interrupt would fire continuously forever, consuming the whole CPU.

The solution is a pair. The **ISR disables** TXEIE when the transmit queue runs dry, and USART_Q_Transmit_NonBlocking **re-enables it** after loading new data.

So the interrupt exists exactly while there is something to send. "**Why must TXEIE be disabled when the queue empties?**" is a very good exam question, and the answer is that TXE is a level condition, not an event.

Listing 6: Book Listings 8.10–8.12. The two access functions and main.

```

1 void USART_Q_Transmit_NonBlocking(Q_T *Q, char *buf, int num_chars) {
2     while (num_chars > 0) {
3         while (Q_Full(Q))
4             ; /* wait for space (see note below) */
5         Q_Enqueue(Q, *buf);
6         buf++; num_chars--;
7     }
8     MODIFY_FIELD(USART2->CR1, USART_CR1_TXEIE, 1); /* re-arm the ISR */
9 }
10
11 void USART_Q_Receive_Blocking(Q_T *Q, char *buf, int num_chars) {
12     while (num_chars > 0) {

```

```

13     while (Q_Empty(&RxQ))
14         ; /* block until data arrives */
15     *buf = Q_Dequeue(&RxQ);
16     num_chars--; buf++;
17 }
18 }
19
20 int main(void) {
21     char buffer[80];
22     uint8_t c;
23     Set_Clocks_To_48MHz();
24     Init_USART2(9600);
25     Q_Init(&TxQ); Q_Init(&RxQ);
26
27     strcpy(buffer, "Hello, world!\n\r");
28     USART_Q_Transmit_NonBlocking(&TxQ, buffer, strlen(buffer));
29
30     while (1) {
31         USART_Q_Receive_Blocking(&RxQ, buffer, 1);
32         c = (uint8_t) buffer[0];
33         sprintf(buffer, "You pressed %c\n\r", c);
34         USART_Q_Transmit_NonBlocking(&TxQ, buffer, strlen(buffer));
35     }
36 }

```

"Non-blocking" here means it **returns after enqueueing the last character, not after the last character is transmitted**. That is the whole win: main carries on immediately while the ISR drains the queue in the background. The book footnotes honestly that it *does* block if the queue lacks space, and that different applications need different handling of that case.

The measured latency between receiving a character and transmitting the response is roughly **150 μ s**.

Book board vs your board

The book's Nucleo routes **USART2 through PA2 and PA3** to a USART inside the ST-LINK debug MCU, which provides a **virtual serial port** to the PC over its USB connection at 9600 baud, 8N1. So "printing to the terminal" is really: your MCU $\rightarrow$ UART $\rightarrow$ ST-LINK MCU $\rightarrow$ USB $\rightarrow$ PC.

Your **STM32F3 Discovery works the same way** but the wiring differs, so check your board's manual for which USART and pins reach the ST-LINK. Your Lab 02 established this and every subsequent lab depends on it.

A useful debugging note the book makes: to watch TX and RX on a logic analyser, connect to the dedicated TX/RX pins on the debug connector rather than the GPIO header pins, because by default the signals may not be routed to the header.

In the lab

Everything you have printed since Lab 02 went through this machinery, and two things are worth reconsidering now that you know how it works.

Your Lab 07 called `HAL_UART_Transmit` inside `HAL_ADC_ConvCpltCallback`. `HAL_UART_Transmit` is the **blocking** variant: it polls TXE for every byte and does not return until the last one is gone. At 9600 baud one byte takes about 1.04 ms, so a twenty-character line takes **roughly 21 ms** spent entirely inside an ISR.

For Lab 07 that was harmless because nothing else needed the CPU. For your self-balancing robot it would be catastrophic: a 21 ms blocking transmit inside an ISR blows a 1 kHz control loop's deadline twenty times over.

The fix is exactly this chapter's structure, and HAL provides it: `HAL_UART_Transmit_IT` or `HAL_UART_Transmit_DMA` enqueue and return immediately. **Use those for any telemetry from a system with a deadline.** And recall Week 07: give the UART interrupt a *lower* priority than the control-loop timer.

8 Tying it back

We asked how two chips exchange data over minimal wires with no shared clock.

Serialization pays a small time cost for a large saving in pins, packages, connectors, cables, and routing, and it turns out to win on speed too, because parallel buses cannot outrun their own skew. Every protocol then answers the same six questions: how to **serialize** (shift registers, at some **baud rate**), how to **time the symbols** (a shared clock, or matched clocks at both ends), how to **frame** a message (a start bit, which is how an asynchronous receiver resynchronises every byte), how to **detect errors** (parity, checksums, CRC), whether to **acknowledge** at all, and how to arbitrate **access and addressing** on shared media.

In software, communication is doubly asynchronous, so a **queue in each direction** decouples hardware timing from program timing: the RX ISR enqueues and a task dequeues, a task enqueues and the TX ISR dequeues. That queue is a **ring buffer** whose index updates are genuine critical sections, and Week 08's lost-update bug is not hypothetical there, it eats received bytes.

UART answers the six questions with: no clock line, a start bit for framing, optional parity, no acknowledgment, and no addressing at all, which is why it is strictly point-to-point. Its baud divisor is $BRR = f_{clock}/\text{baud}$, its five frame settings must be agreed out-of-band, and its transmit interrupt must disable itself when the queue empties because TXE is a level, not an event.

Next week, SPI: add the clock wire back and see what you buy with it.

Cheat sheet: Week 14

Why serial

Parallel = all parts simultaneously available, one wire per bit. **Serial** = sequentially available.

Serial saves: **pin count**, **package size**, **connector contacts**, **cable wires**, **board routing**.

And it is faster in practice, because parallel buses cannot outrun **skew** at high frequency. (PATA→SATA, PCI→PCIe.)

OSI layers: **1 physical** (voltages), **2 data link** (MAC = when to transmit, LLC = framing and errors), **3 network** (addressing).

The six problems every protocol solves

1. **Serialization** — shift registers, PISO out and SIPO in. **Baud rate** = symbol rate.
2. **Symbol timing** — sample **once per symbol, in the middle**. **Synchronous** sends a clock (3 wires, reliable); **asynchronous** does not (2 wires, clocks must match).
3. **Framing** — a **framing symbol** (start bit) marks the message start and starts the receiver's clock.
4. **Error detection** — parity, checksum, CRC.
5. **Acknowledgment** — in-message, separate message, or **none at all**.
6. **MAC and addressing** — rules preventing **collisions**; commonly **master/slave**. Address via **select lines** (SPI) or **in the message** (I²C).

Software structure

Communication is **doubly asynchronous**: you can't predict when data arrives, *or* when the transmitter is next ready.

RX: ISR reads peripheral → **enqueues** → task dequeues later.

TX: task **enqueues** → ISR dequeues → writes peripheral.

Two queues. The queue **decouples hardware timing from software timing**.

Ring buffer: array + Head (oldest) + Tail (next free) + Size, indexes wrapping to zero.

Make the size a power of two so wrapping is a mask, not a modulus.

Declare queues **volatile**. **Protect index/size updates with a critical section** — the Week 08 lost-update bug here *eats received bytes*. Keep the section short: the array write goes outside it.

Never block inside an ISR — it adds sporadic, unreproducible latency. **Drop the data and flag an error instead**.

UART frame and baud rate

Start (0) · **Data** (typically 8) · **Parity** (optional) · **Stop (1)**.

Parity counts ones across **data and parity bit together**. Detects any **odd** number of errors, **misses every even** number.

A zero where a stop bit belongs = **framing error**, usually a baud-rate mismatch.

Oversampling: receiver samples each bit multiple times and takes the majority. **16× by default**; OVER8=1 makes it 8.

$$\text{BRR} = \frac{f_{\text{BusClock}}}{\text{baud}} \quad 48 \text{ MHz}, 9600 \text{ baud} \Rightarrow \text{BRR} = 5000$$

BRR is an integer, so most rates are approximate. 115200 at 48 MHz → BRR= 416 →

actual 115,385 baud, 0.16% error. **Tolerance is about $\pm 2\%$ total.**
Consistent garbage = wrong baud rate. Silence = nothing transmitting. Different bugs.

USART registers and flags

CR1: M1/M0 data length · PCE parity enable · PS even/odd · TE/RE/UE enables · TXEIE/TCIE/RXNEIE/PEIE · OVER8.

CR2: STOP (0.5/1/1.5/2), MSBFIRST. CR3: EIE. BRR: divisor.

TDR write $\Rightarrow$ **clears** TXE. RDR read $\Rightarrow$ **clears** RXNE.

Flags in ISR: TXE room to send · TC complete · RXNE data ready · IDLE · PE parity · FE framing · ORE overrun · NF noise. Cleared by writing 1 to ICR.

ORE **means your software was too slow**, not that the wiring is wrong. On some parts it **blocks further reception until cleared**.

8N1 = 8 data, No parity, 1 stop. **Five settings must match at both ends and none is negotiated.**

The interrupt-driven pattern

RX branch: read RDR, enqueue **if there is room, else drop**.

TX branch: if the queue has data, dequeue into TDR; **else clear** TXEIE.

Why TXEIE must self-disable: TXE is a **level, not an event** — it is set whenever the buffer is empty, which is almost always. Leaving the interrupt enabled would fire it forever. The transmit function **re-enables** TXEIE after loading new data.

"Non-blocking" = returns after *enqueueing*, not after transmitting.

HAL warning: HAL_UART_Transmit **blocks** — at 9600 baud, 20 characters is ≈ 21 ms. Never inside an ISR with a deadline. Use HAL_UART_Transmit_IT or _DMA.

Likely exam prompts from this section

- Compute BRR for a given clock and baud rate, and the resulting error. *Near certain.*
- Why serialize, given parallel is wider? Include **skew**.
- Draw and label a UART frame; explain the purpose of the start bit.
- Distinguish synchronous from asynchronous communication.
- Why does interrupt-driven communication need a queue?
- Why must queue index updates be in a critical section?
- Why must TXEIE be disabled when the transmit queue empties?
- What does an overrun error tell you, and how do you fix it?